AI ASSISTED CODING

Assignment-9

2303A51463

B-07

Documentation Generation – Automatic Documentation and Code Comments

Task 1: Basic Docstring Generation

Prompt:

Generate a Google-style docstring for a Python function that returns the sum of even numbers and sum of odd numbers from a list of integers.

[1]

```
def sum_even_odd(numbers):
    """
    Calculates the sum of even and odd numbers in a list.

    Args:
        numbers (list): A list of integers.

    Returns:
        tuple: A tuple containing:
            - sum_even (int): Sum of all even numbers.
            - sum_odd (int): Sum of all odd numbers.

    Raises:
        TypeError: If the input is not a list.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    sum_even = 0
    sum_odd = 0
    for num in numbers:
        if num % 2 == 0:
            sum_even += num
        else:
            sum_odd += num
    return sum_even, sum_odd
print(sum_even_odd([1,2,3,4,5,6]))
```

```
(12, 9)
```

Task 2: Automatic Inline Comments

Prompt:

You are a Python documentation expert.

I have written a student management class named sru_student with:

- Attributes: name, roll_no, hostel_status
- Methods: fee_update(amount) and display_details()

Add meaningful inline comments to the entire code.

Explain each logical block clearly.

Do not add unnecessary or redundant comments.

Ensure comments are simple and easy for beginners to understand.

Return the fully commented Python class.

[2]

```python
class sru_student:
    # Constructor to initialize student details
    def __init__(self, name, roll_no, hostel_status):
        self.name = name              # Store student name
        self.roll_no = roll_no        # Store roll number
        self.hostel_status = hostel_status  # Store hostel status (True/False)

    def fee_update(self, amount):
        self.fee = amount
    # Method to display student details
    def display_details(self):
        print("Name:", self.name)     # Print student name
        print("Roll No:", self.roll_no)  # Print roll number
        print("Hostel:", self.hostel_status)  # Print hostel status
        print("Fee:", getattr(self, 'fee', 0))  # Print fee (default 0 if not set)
student1 = sru_student("Harshitha", 101, True)
student1.fee_update(50000)
student1.display_details()
```

```
Name: Harshitha
Roll No: 101
Hostel: True
Fee: 50000
```

Task 3: Module-Level and Function-Level Documentation

Prompt:

Generate a NumPy-style module-level and function-level docstrings for a calculator module containing add, subtract, multiply, divide functions.

[7]

```
def add(a, b):
    """
    Add two numbers.
    Parameters
    a : int or float
        First number.
    b : int or float
        Second number.
    Returns
    int or float
        Sum of a and b.
    """
    return a + b
def subtract(a, b):
    """Subtract two numbers.
    Parameters
    a : int or float
        First number.
    b : int or float
        Second number.
    Returns
    int or float
        Difference of a and b.
    """
    return a - b

def multiply(a, b):
    """Multiply two numbers.
    Parameters
    a : int or float
        First number.
    b : int or float
        Second number.
```

```
        b : int or float
            Second number.
        Returns
        int or float
            Product of a and b.
        """
        return a * b

    def divide(a, b):
        """Divide two numbers.
        Parameters
        a : int or float
            Numerator.
        b : int or float
            Denominator.
        Returns
        float
            Result of division.
        Raises
        ZeroDivisionError
            If b is equal to zero."""
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return a / b

    print("Add:", add(10, 5))
    print("Subtract:", subtract(10, 5))
    print("Multiply:", multiply(10, 5))
    print("Divide:", divide(10, 5))
```

```
Add: 15
Subtract: 5
Multiply: 50
Divide: 2.0
```